

Quantum Sim Platform — Đặc tả Kernel

Quantum Device Simulation Platform — Kernel Specification

Version: v 0 . 1 (draft) · **Date:** 2 0 2 6 - 0 5 - 2 6

Product: open-source, device-level, physically-realistic simulation platform (“a virtual quantum bench”) for quantum-technology R&D — aimed at low-budget researchers & students, especially in developing countries. **Validation domains (in order):** (1) QKD → (2) quantum sensing → (3) quantum-computing hardware. QKD is the first *reference design*; QRNG rides along as a near-free early plugin.



Cách đọc: đây là đặc tả kỹ thuật của **kernel** (lỗi trung lập), tách khỏi các thư viện theo lĩnh vực. Mỗi mục có khối “**Giải thích**” tiếng Việt cho các quyết định kiến trúc quan trọng. Mục tiêu của kernel: cho phép **lắp thiết bị từ các khối, giảm impairment thật, chạy mô phỏng đa thang thời gian, rồi tinh chỉnh tham số và quan sát kết quả theo thời gian** — đúng cái “Lớp 3” mà bench thật dạy, nhưng làm được ảo, miễn phí.

0 . Design principles (the non-negotiables)

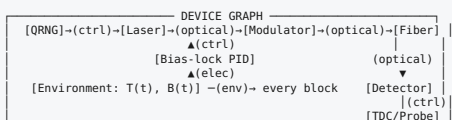
- 1 . **Device-level, not protocol-level.** We model the *physical box and its impairments*, not an abstract key-rate formula. The unit of value is *practicing the tuning loop*.
- 2 . **Domain-agnostic kernel + per-domain plugin libraries.** Ship `kernel + QKD` first; add sensing, QC-hardware, ... as plugins (incl. community). Never hard-code QKD assumptions into the kernel.

- 3 . **Behavioral / multi-rate, not first-principles.** No Maxwell/ Schrödinger PDE solving over hours. Calibrated behavioral models coupled across timescales. Fidelity ceiling = calibration quality.
- 4 . **Calibration is first-class.** Every impairment parameter is sourced (datasheet / paper DOI / measurement) and the platform can *validate against a published experiment*.
- 5 . **Accessible.** Python-first, free dependencies, runs on a laptop, Jupyter-driven.
- 6 . **Reuse, don't reinvent.** QuTiP for open-system quantum dynamics; ngspice for analog; GHDL/ Verilator for HDL.

Giải thích: 6 nguyên tắc này là “hiến pháp”. Quan trọng nhất là (2) và (3): **kernel phải trung lập** (không biết gì về QKD) và **mô hình hóa ở mức behavioral** (không giải phương trình vật lý gốc) — đây là hai quyết định cho phép vừa *tổng quát* vừa *chạy đủ nhanh trên laptop*.

1 . Core idea: a device is a typed-signal block graph + a multi-rate engine

A simulated device is a **directed graph**: - **Nodes = Blocks** — physical/functional units (laser, modulator, fiber, detector, qubit, atomic vapor cell, PID loop, FPGA, ...). - **Edges = typed Signals** — what flows between blocks (light, voltage, a quantum state, a control setpoint, temperature, classical bits). - A **multi-rate scheduler** advances the whole graph across wildly different timescales (ps → hours).



Giải thích: mô hình “đồ thị khối + tín hiệu có kiểu” chính là **interface § 2 trong file kiến trúc QKD** được tổng quát hóa. Một thiết bị bất kỳ (QKD, từ kế, chip qubit) đều biểu diễn được dưới dạng này — chỉ khác *loại khối* và *loại trạng thái lượng tử* chạy trên cạnh.

2 . Signal types (typed ports)

Connections are **type-checked at graph-build time**. Core signal types:

Signal type	Carries	Used by
OpticalSignal	optical mode structure (time-bins, polarization, frequency) + a QuantumState or classical field envelope	QKD, sensing (probe/pump light), photonic QC
ElectricalSignal	analog voltage/current waveform (time-domain)	drivers, bias, SPICE co-sim, readout
QuantumState	abstract quantum info carrier — see § 3	all domains
ControlSignal	digital/logical: triggers, clock, DAC setpoints, gate patterns	FPGA, control loops
Environmental	scalar/vector fields: temperature, vibration, magnetic field B(t)	global drift, sensing
ClassicalData	bits / messages / frames	post-processing, networking

Giải thích: tách OpticalSignal (vật mang) khỏi QuantumState (thông tin lượng tử bên trong) là then chốt: cùng một sợi quang có thể mang trạng thái Fock (QKD) hay trường cổ điển (bơm laser cho sensing). Environmental là tín hiệu *toàn cục* (nhiệt độ, từ trường) — chính nó tạo ra “trôi” mà việc tinh chỉnh phải chống lại, và với sensing thì từ trường B(t) **là đại lượng cần đo**.

3 . Pluggable QuantumStateBackend — the generalization lever

The single most important abstraction. Blocks declare which backend they operate on; the kernel treats them uniformly. Domain boundaries (e.g. light ↔ atom) implement **backend converters**.

Backend	Representation	Primary domain	Library
FockBackend	photon-number / detection-event Monte Carlo (faint pulses)	QKD (DV), QRNG	custom + NumPy
CoherentStateBackend	Gaussian / quadratures	CV-QKD, optical sensing	NumPy / strawberryfields-style
BlochEnsembleBackend	spin/Bloch vectors + T ₁ /T ₂ relaxation, optical pumping	quantum sensing (magnetometer, atomic clock)	QuTiP
DensityMatrixBackend	ρ + Lindblad master equation	QC hardware (qubits)	QuTiP

```
class QuantumStateBackend(Protocol):
    def init_state(self, spec) -> State: ...
    def evolve(self, state: State, op, dt: float) -> State: ... # unitary + dissipative
    def measure(self, state: State, observable) -> Outcome: ...
    def sample_event(self, state: State, model) -> Event | None: ... # e.g. photon click
    def to(self, other: "QuantumStateBackend", state) -> State: ... # boundary conversion
```

Giải thích — vì sao đây là “đòn bẫy”: mày chốt **QC-hardware là lĩnh vực thứ 3**, nên kernel **không được** giả định trạng thái lượng tử = “số photon”. Phải có *nhiều backend cấm-thay*: Fock cho QKD, Bloch cho sensing, density-matrix/Lindblad cho qubit. Nếu thiết kế đúng cái interface này ngay từ đầu thì sau này nhét QC-hardware vào chỉ là *thêm một backend*, không phải đập kernel. **QuTiP** cho ta gần như miễn phí hai backend khó nhất (Bloch + Lindblad) → hạ rủi ro “mở rộng”.

4 . The Block model

```
class Block(Protocol):
    ports_in: dict[str, SignalType]
    ports_out: dict[str, SignalType]
    params: Params # tunable (the knobs you “tinh chỉnh”)
```

```

impairments: list[Impairment] # $5
calibration: CalibrationProfile # $7
timescale: Timescale # natural rate → tells scheduler how to drive it
fidelity_level: int # 0=ideal, 1=behavioral, 2=SPICE/HDL co-sim

def react(self, t, inputs) -> outputs: ... # event-driven (a pulse arrives)
def step(self, t, dt) -> None: ... # stepped (continuous drift / control)

```

- **Selectable fidelity levels:** an `ideal` model for fast first-pass, a `behavioral` model with impairments, and (where it matters) a `co-sim` level that delegates to ngspice (analog board) or GHDL/Verilator (firmware). Same ports → swap fidelity without rewiring.

Giải thích: mỗi khối có “núm vặn” (`params`) — đây chính là cái người học sẽ tinh chỉnh. Và mỗi khối có **nhiều mức độ trung thực**: chạy nhanh kiểu lý tưởng để dựng hệ, rồi bật mức SPICE cho đúng board gating khi cần soi chi tiết — *cùng cổng, đổi ruột*.

5 . Impairment model interface

Behavior = ideal model ◦ composable impairments. Each impairment declares its **timescale** so the scheduler treats it correctly.

Impairment class	Examples	Timescale
Static	insertion loss, finite extinction ratio, η	constant
Fast-stochastic	jitter, shot noise, dark counts	per-event
Stateful-history	afterpulsing (non-Markovian), dead time, qubit leakage	event + memory
Slow-drift	bias drift, AMZI phase drift, laser λ drift, T_1/T_2 fluctuation, $1/f$	ms → hours (OU process)

```

class Impairment(Protocol):
    timescale: Timescale
    params: Params
    def apply(self, signal_or_state, t, ctx): ... # transforms output

```

Giải thích: đây là nơi “giống thật” sống. Ví dụ **afterpulse** không phải nhiễu ngẫu nhiên đơn giản — nó *nhớ lịch sử* (non-Markovian), nên là

Stateful-history. **Trôi pha/bias** là Slow-drift (mô hình Ornstein-Uhlenbeck) chạy ở thang giây-giờ. Phân loại theo thang thời gian để scheduler (§ 6) biết *khi nào* phải tính lại từng thứ → vừa đúng vừa nhanh.

6 . Multi-rate simulation engine (the hard core, highest risk)

The problem: photonics ~**ps**, gating ~**ns**, control loops ~**μs-ms**, thermal/aging drift ~**s-hours**, key/measurement accumulation ~**minutes**. Cannot brute-force ps-resolution for hours.

Approach — hybrid, FMI-co-simulation-inspired:

- 1 . **Event-driven fast layer.** Optical pulses & analog transients are *events* carrying a waveform/state payload. A block runs `react()` only when an event arrives — no idle ps-stepping.
- 2 . **Stepped slow layer.** Drift & control loops integrate on a coarse clock (ms-s) via `step()`, updating *quasi-static parameters* that modulate the fast layer (e.g. current AMZI phase, current detector bias). Fast blocks read the latest slow-state when an event passes.
- 3 . **Statistical aggregation.** Don't simulate all 10^9 pulses individually — simulate representative *batches* + Monte Carlo, accumulate metrics statistically, while slow-state evolves between batches. This is what makes “hours of operation” tractable on a laptop.
- 4 . **External co-sim slaves (FMI-like).** ngspice / GHDL / Verilator wrapped behind `do_step()`; the orchestrator coordinates time and exchanges port values.

slow clock ———— (ms-s: drift, PID, batch boundary)
| updates θ_{phase} , V_{bias} , T_1 ...

Giải thích: đây là **trái tim và rủi ro lớn nhất**. Ý tưởng: *tách thang thời gian*. Cái nhanh (xung) chạy theo **sự kiện** (chỉ tính khi có xung), cái chậm (trôi nhiệt, vòng điều khiển) chạy theo **bước thô** rồi *điều biến* cái nhanh. Và thay vì mô phỏng cả tỉ xung, ta mô phỏng **theo lô + thống kê**. Nhờ vậy “chạy thiết bị suốt vài giờ” mới khả thi trên laptop. **Prototype đầu tiên phải chứng minh đúng cái cơ chế này** (xem § 1 3 , M 0).

7 . Calibration & validation framework

- **Calibration profiles** (YAML/JSON): block/impairment params with **provenance** tags (`datasheet:` / `doi:` / `measured:`). Ship a library of profiles from public datasheets & papers.
- **Validation harness:** encode a *published experiment* (setup graph + params) → run → compare sim metrics to published results → report match. Credibility comes from this (qkdSim earned trust by matching a real B 9 2 experiment).
- **Provenance & uncertainty:** every parameter carries a source and optional error bar; metrics can propagate uncertainty.

Giải thích: không có thiết bị thật thì calibrate theo **datasheet + paper** (có rất nhiều). Và để cộng đồng tin, nền tảng phải **tái hiện được ít nhất một thí nghiệm đã công bố** cho mỗi domain. Đây là điều kiện sống còn của uy tín — không có nó thì sim chỉ là đồ chơi.

8 . Control / firmware layer

- Discrete-time **control blocks**: PID, lock-in amplifier, bias-lock, phase-lock — read sensor ports, drive setpoints at their own rate (slow layer).
- Optional **HDL co-sim** (GHDL/Verilator) to run *real firmware* against the simulated plant — hardware-in-the-loop, but virtual.

Giải thích: vòng điều khiển (khóa pha/bias) chính là thứ giữ thiết bị ổn định — và là một phần lớn của “tinh chỉnh”. Cho phép nạp **firmware FPGA thật** chạy ngược lại mô hình vật lý ảo = luyện đúng kỹ năng thật mà không cần phần cứng.

9 . Observability: probes, metrics, the tuning loop

- **Probes** attach to any port → time-series (this is your “oscilloscope/counter”).
- **Metrics** are domain-plugin-provided:
 - QKD: QBER, secret-key-rate, visibility, extinction ratio, afterpulse prob.
 - Sensing: sensitivity ($T/\sqrt{\text{Hz}}$), Allan deviation, bandwidth.
 - QC-hw: gate fidelity, T_1/T_2 , readout fidelity, crosstalk.
- **Sweep / optimize harness**: vary params → metric surface; this is the tuning UX.
- **Scenario files**: declarative graph + params + run + metrics, so experiments are shareable.

Giải thích: đây là mặt người dùng của “tinh chỉnh”: gắn probe → vận núp → xem QBER/độ nhạy/độ trung thực phản ứng theo thời gian.

Quét tham số để thấy *bề mặt độ nhạy* — đúng cái mà một nghiên cứu sinh cần “cảm” được trước khi đụng phần cứng đắt tiền.

1 0 . Plugin / package architecture

qsim-core
qsim-qkd
qsim-sensing
qsim-qchw
qsim-qrng

→ graph, signals, scheduler, backends API, impairment API, calibration, probes
→ blocks (laser, MZM, AMZI, SPAD...), Fock backend, QKD metrics, BB84 reference design
→ blocks (vapor cell, pump/probe, coils...), Bloch backend, magnetometer metrics
→ blocks (qubit, microwave drive, resonator readout...), DensityMatrix/Lindblad backend
→ trivial early plugin (reuses qkd source+detector) – proves modularity cheaply

Plugins register block types, state backends, metrics, and calibration profiles via an entry-point registry. **The kernel never imports a plugin.**

1 1 . Domain coverage matrix (proves the kernel is general, not QKD-shaped)

Kernel feature	QKD	Sensing	QC-hardware
State backend	Fock	Bloch ensemble	Density-matrix/ Lindblad
Dominant signal	Optical+Fock	Environmental B(t) + probe light	Control pulses + Quantum state
Backend converter exercised	—	light ↔ atom (optical pumping)	pulse ↔ p (drive Hamiltonian)
Stateful impairment	afterpulsing	spin relaxation T ₂	qubit leakage / crosstalk
Slow-drift	AMZI phase, λ	bias field, temperature	T ₁ / T ₂ fluctuation, 1 / f flux noise
Multi-rate span	ps pulse ↔ min key accum.	μs probe ↔ s Allan	ns gate ↔ μs readout ↔ drift
Key metric	QBER / SKR	sensitivity / Allan dev	gate & readout fidelity

Giải thích: bảng này là **bài kiểm tra tính tổng quát**. Nếu cùng một kernel chạy được cả 3 cột (mỗi cột ép một backend + một loại impairment + một dải thang thời gian khác nhau) thì abstraction đã

đủ rộng. Cột QC-hardware ép DensityMatrixBackend và “pulse↔p”
— chính vì vậy ta thiết kế nó ngay từ đầu thay vì để sau.

1 2 . Tech stack & reuse

Need	Choice	Why
Core language	Python 3.11+	accessible, free, ecosystem
Numerics	NumPy / SciPy	standard
Open-system quantum dynamics	QuTiP	gives Bloch + Lindblad backends nearly free (de-risks sensing & QC-hw)
Analog co-sim	ngspice (via subprocess / PySpice)	the detector/driver boards (already in QKD toolchain)
Firmware co-sim	GHDL / Verilator	run real HDL against virtual plant
Event engine	custom (SimPy-inspired) + multi-rate orchestrator	the FMI-like core
UX	Jupyter notebooks first → web GUI later	fast to ship, accessible
Viz	matplotlib / plotly	—

Giải thích: **QuTiP** là quân bài lớn: nó là chuẩn mở cho động học hệ lượng tử mở → cho ta backend Bloch (sensing) và Lindblad (qubit) gần như miễn phí, biến lời hứa “mở rộng đa lĩnh vực” từ rủi ro thành khả thi. Ta chỉ phải tự viết phần *thực sự chưa ai làm*: **engine multi-rate** và **mô hình impairment device-level + khung calibrate**.

3 . Roadmap (QKD-first)

Milestone	Deliverable	Proves
M 0 — Hard-core slice	kernel skeleton + multi-rate scheduler + ONE vertical: faint pulse → fiber → InGaAs detector (afterpulse/dark/ dead-time/jitter) under a gate clock, with a slow AMZI-phase drift coupled in, QBER accumulating by batches	the multi-rate engine works & is fast on a laptop (the # 1 risk)
M 1 — QKD reference design	full decoy-BB 8 4 Alice+Bob from docs/01, end-to-end	the platform models a real device; validate vs a published QKD experiment
M 2 — Productize the loop	calibration framework + scenario files + sweep/optimize + Jupyter UX + qrng plugin	the tuning UX + modular plugins
M 3 — Sensing plugin	atomic magnetometer (Bloch backend, light↔atom converter, B(t))	kernel generalizes beyond comms; validate vs published sensitivity/Allan
M 4 — QC-hardware plugin	1 – 2 superconducting/ion qubit control+readout (Lindblad backend)	kernel handles density-matrix; validate gate fidelity vs published

Cross-cutting: packaging, docs, education materials, community-contribution model (it’s open-source).

Giải thích: M 0 **trước tiên và quan trọng nhất** — nó tấn công thẳng rủi ro lớn nhất (engine multi-rate) trên một lát cắt nhỏ nhưng đầy đủ “chất khó” (impairment động + trôi chậm + tích lũy theo lô). Nếu M 0 chạy được và nhanh trên laptop thì cả nền tảng khả thi. M 1 lấy uy tín bằng cách tái hiện thí nghiệm thật. M 3 /M 4 mới là lúc chứng minh “đã lĩnh vực”.

4 . Key technical risks / open problems

- 1 . **Multi-rate coupling — correctness & speed** (the make-or-break). Mitigate: prototype in M 0 ; benchmark against a brute-force reference on a small case.

- 2 . **Backend conversion at domain boundaries** (light ↔ atom, pulse ↔ p) — physically faithful & lossless enough. Mitigate: lean on QuTiP-validated interaction Hamiltonians; validate per domain.
 - 3 . **Calibration data availability & the fidelity ceiling** — sim is only as real as its profiles. Mitigate: provenance tags + published-experiment validation; be honest about un-modeled effects.
 - 4 . **Statistical-aggregation accuracy** for batch/Monte-Carlo key accumulation vs full simulation. Mitigate: convergence tests; expose batch size as a fidelity/speed knob.
 - 5 . **Scope discipline** — “all quantum tech” can balloon. Mitigate: kernel never imports plugins; ship QKD-first; new domains land as plugins only.
-

1 5 . Immediate next step

Build **M 0** as a runnable proof-of-concept: minimal `qsim-core` (graph + signals + multi-rate scheduler + Fock backend + impairment API) and the single QKD vertical slice (pulse → fiber → gated InGaAs detector with realistic impairments + slow phase drift → QBER by batches). If that runs and stays fast, the platform is real.